
GitLab CI/CD Pipeline Infrastructure

Global Workflow Technical Reference

NOAA / NWS / NCEP / EMC
Global Workflow Development Team
<https://github.com/NOAA-EMC/global-workflow>

March 2026

Abstract

This document provides a comprehensive, standalone technical reference for the GitLab CI/CD pipeline infrastructure used by the NOAA/EMC global-workflow project. It covers the repository mirroring strategy between GitHub and GitLab, the pipeline architecture and configuration, the GitLab runner deployment on NOAA's Research and Development High-Performance Computing Systems (RDHPCS), nightly pipeline operations, and the day-to-day maintenance procedures that keep the system operational. All architectural diagrams are reproduced as native vector figures for print-quality output.

Contents

1	Overview	2
1.1	Key Design Principles	2
2	Repository Mirroring: GitHub to GitLab	3
2.1	Pull Mirroring (Licensed GitLab Instance)	3
2.2	Push Mirroring (Community GitLab at VLab)	4
2.3	Mirror Chain Summary	5
3	Pipeline Architecture	5
3.1	Pipeline Stages	5
3.2	Pipeline Modalities	6
3.2.1	PR Cases (PIPELINE_TYPE=pr_cases)	6
3.2.2	CTests (PIPELINE_TYPE=ctests)	7
3.3	Per-Host Test Matrices	7
3.4	Pipeline Variables	8

4	GitHub Actions Integration	9
4.1	Triggering a Pipeline	9
4.2	Required GitHub Secrets and Variables	10
4.3	PR Label Lifecycle	10
5	Nightly Pipeline Operations	11
5.1	Directory Management	11
5.2	Badge Updates	11
6	GitLab Runner Setup	11
6.1	Platform Configuration Files	12
6.2	The <code>launch_gitlab_runner.sh</code> Script	12
6.2.1	Setup Prerequisites	13
6.2.2	Registering a Runner	13
6.2.3	Starting a Runner	13
6.2.4	Unregistering a Runner	14
6.3	Runner Directory Layout	14
6.4	Runner Maintenance	14
7	Pipeline Execution Details	15
7.1	Build Stage	15
7.2	Test Execution (PR Cases)	15
7.3	Test Execution (CTests)	16
7.4	Finalize Stage	16
8	Adding a New Host Platform	16
9	File Reference	17
10	Troubleshooting	18
10.1	Runner Not Picking Up Jobs	18
10.2	Build Failures	18
10.3	Test Case Timeouts	18
10.4	CTest Baseline Mismatches	19
10.5	GitLab Mirror Sync Issues	19

1 Overview

The global-workflow CI/CD system uses **GitLab CI/CD** as the execution engine for continuous integration testing across NOAA’s Research and Development High-Performance Computing Systems (RDHPCS). GitHub remains the **authoritative repository** where all development, code review, and pull request activity occurs.

The fundamental challenge this infrastructure solves is that NOAA’s HPC systems (Hera, Gaea, Orion, Hercules, Ursa) are not directly accessible from GitHub Actions runners. By mirroring the repository to GitLab and placing GitLab runners directly on those HPC systems, the project gains the ability to build and test the workflow in the same environments where it will be deployed operationally.

1.1 Key Design Principles

- **GitHub is authoritative** — All development happens on GitHub (<https://github.com/NOAA-EMC/global-workflow>). GitLab is used solely as a CI execution platform.
- **Two-tier mirroring** — A licensed GitLab instance performs the pull mirror from GitHub, and subsequently push mirrors to the NOAA community GitLab instance.
- **HPC-native testing** — Runners execute directly on the target HPC nodes, ensuring tests build and run against the real Spack-Stack software environment.
- **Multi-modal pipelines** — The system supports both comprehensive end-to-end experiment cases and fast CTest-based functional checks.
- **GitHub feedback loop** — Pipeline results flow back to GitHub through PR labels, PR comments (including error log gists), and status badges.

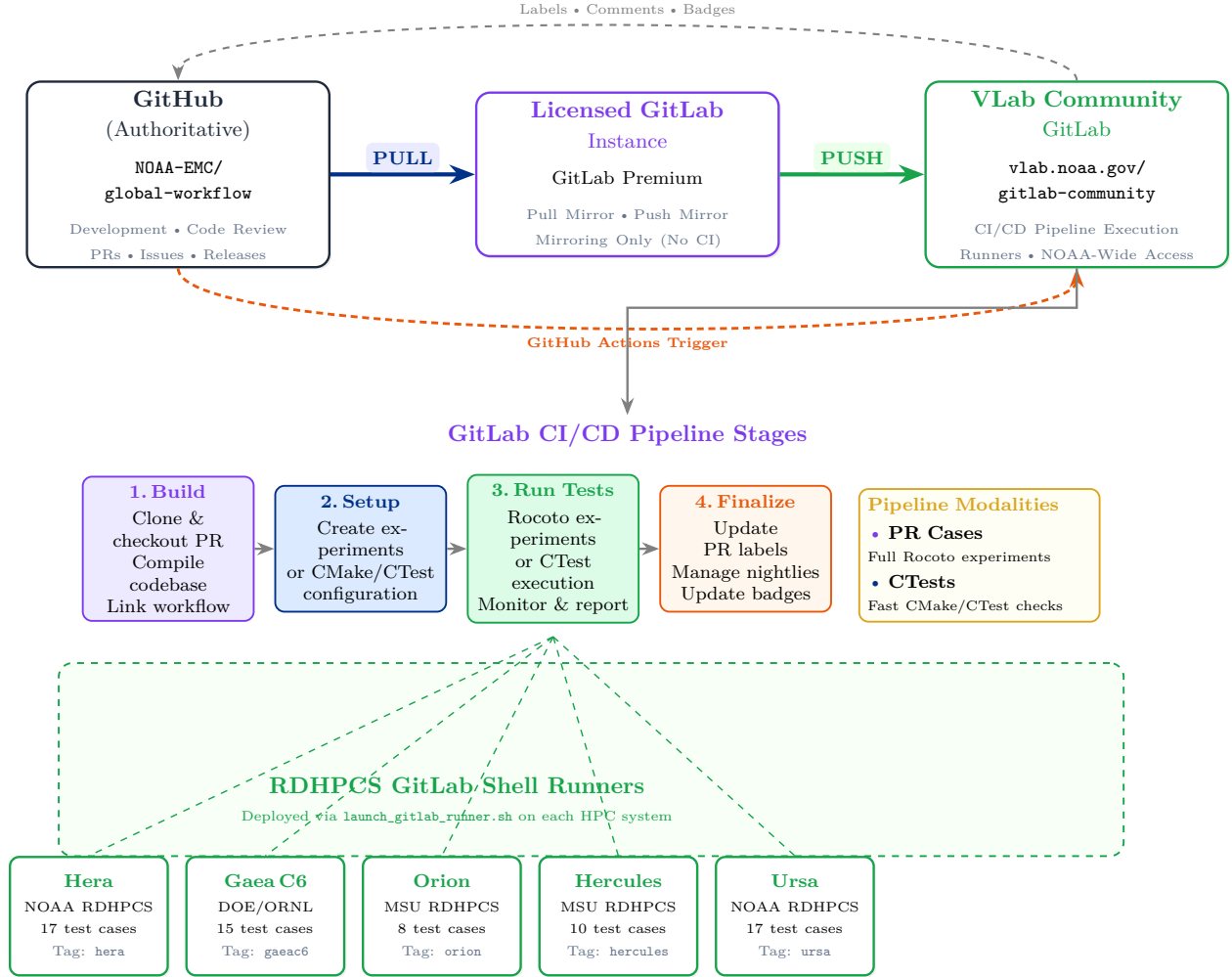


Figure 1: High-level CI/CD architecture showing repository mirroring, pipeline stages, and RDHPCS runner deployment.

2 Repository Mirroring: GitHub to GitLab

Because GitHub is the authoritative source of truth and GitLab is the CI execution platform, a reliable synchronization mechanism is required. The global-workflow project uses a **two-stage mirroring strategy** involving two GitLab instances.

2.1 Pull Mirroring (Licensed GitLab Instance)

The first stage uses **pull mirroring**, a feature that is only available on licensed (paid) tiers of GitLab (Premium or Ultimate). A single licensed GitLab instance is configured to pull from the authoritative GitHub repository:

Table 1: Pull Mirror Configuration

Setting	Value
Source repository	https://github.com/NOAA-EMC/global-workflow.git
Direction	Pull
Scope	All branches
Sync frequency	Automatic (every few minutes)

The licensed instance’s sole purpose is **mirroring** — it does not run any CI/CD pipelines itself. Its pull mirror keeps the GitLab copy synchronized with GitHub, and its push mirror (described below) propagates changes onward.

Note

Pull mirroring is an **advanced feature** available only on licensed instances of GitLab (Premium tier and above). It is not available on GitLab Community Edition (CE) or the free tier. This is why a separate licensed instance is required for the first stage of the mirror chain.

2.2 Push Mirroring (Community GitLab at VLab)

The second stage uses **push mirroring** from the licensed GitLab instance to the NOAA community GitLab instance hosted at VLab:

Table 2: Push Mirror Configuration

Setting	Value
Target repository	https://vlab.noaa.gov/gitlab-community/NWS/Operations/NCEP/EMC/global-workflow.git
Direction	Push
Scope	All branches
Sync frequency	Automatic (every few minutes)

The VLab community GitLab instance is where the **CI/CD pipelines actually execute**. GitLab runners deployed on RDHPCS systems register against this instance, and all pipeline stages (build, setup, test, finalize) run here. This instance also provides the broader NOAA user community with read access to the repository.

2.3 Mirror Chain Summary

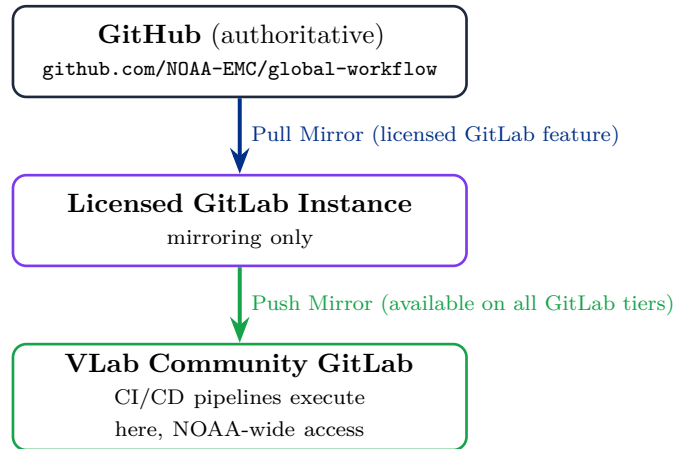


Figure 2: Complete mirror chain from GitHub to the CI/CD execution environment.

Both mirrored repositories track **all branches**, ensuring that any branch pushed to GitHub (including PR branches fetched during pipeline execution) is available for CI testing.

Important

Developers should **never push directly** to either GitLab instance. All code changes must flow through GitHub. The GitLab mirrors are read-only copies maintained by the mirroring configuration.

3 Pipeline Architecture

The pipeline is defined across four YAML configuration files that are included from the top-level `.gitlab-ci.yml`:

Table 3: Pipeline Configuration Files

File	Purpose
<code>.gitlab-ci.yml</code>	Main orchestration: stages, variables, base templates, build template
<code>dev/ci/gitlab-ci-cases.yml</code>	Templates for standard experiment test cases (setup, run, finalize)
<code>dev/ci/gitlab-ci-ctests.yml</code>	Templates for CTest-based functional testing (CMake/CTest)
<code>dev/ci/gitlab-ci-hosts.yml</code>	Host-specific jobs, test matrices, runner tags, and conditional rules

3.1 Pipeline Stages

Every pipeline execution proceeds through four stages in order:

1. **build** — Clone the repository, checkout the PR branch (if applicable), build the codebase via `ci_utils.sh build`, and link the workflow.
2. **setup_tests** — Prepare the test environment: create experiment directories (PR Cases) or configure the CMake/CTest build (CTests).
3. **run_tests** — Execute the tests: run Rocoto-orchestrated experiments (PR Cases) or run `ctest` with specific labels (CTests).
4. **finalize** — Report results: update GitHub PR labels, manage nightly directory symlinks, and update status badges.

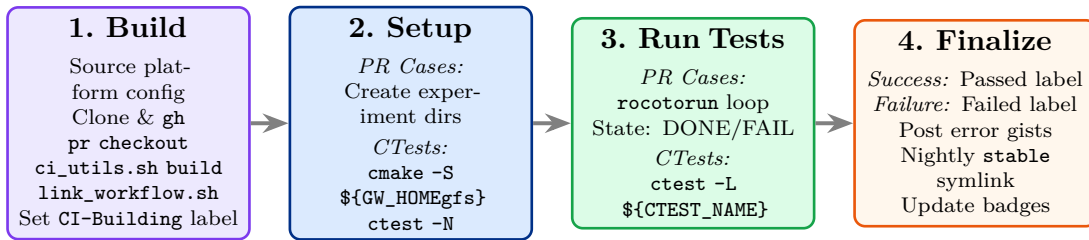


Figure 3: Detailed view of the four pipeline stages and their key operations.

3.2 Pipeline Modalities

The `PIPELINE_TYPE` variable controls which testing modality runs.

3.2.1 PR Cases (`PIPELINE_TYPE=pr_cases`)

Comprehensive end-to-end experiment testing. Each test case is defined by a YAML file in `dev/ci/cases/pr/` that specifies an experiment configuration:

Listing 1: Example PR case definition (`dev/ci/cases/pr/C48_ATM.yaml`)

```
# Example: dev/ci/cases/pr/C48_ATM.yaml
experiment:
  net: gfs
  mode: forecast-only
  app: ATM
  resdetatmos: 48
  idate: 2021032312
  edate: 2021032312

workflow:
  engine: rocoto
  rocoto:
    maxtries: 2
```

The pipeline creates a full experiment directory, launches Rocoto, and monitors the workflow to completion. Failures are detected through Rocoto state tracking and reported back to the GitHub PR with error log gists.

Currently defined PR case tests include:

- C48_ATM — Atmosphere-only forecast
- C48_S2SW — Coupled atmosphere-ocean-ice-wave
- C48_S2SWA_gefs — GEFS ensemble coupled run
- C48mx500_3DVarA0WCDA — 3DVar coupled data assimilation
- C48mx500_hybA0WCDA — Hybrid EnVar coupled data assimilation
- C96C48_hybatmDA — Hybrid atmosphere-only data assimilation
- C96C48_hybatmsnowDA — Hybrid atmosphere + snow DA
- C96C48_hybatmsoilDA — Hybrid atmosphere + soil DA
- C96_atm3DVar — C96 resolution 3DVar atmosphere
- C96_gcafs_cycled — GCAFS cycled system
- C96mx100_S2S — Seasonal-to-subseasonal coupled
- C48_gsienkf_atmDA — GSI ensemble Kalman filter
- C48_ufsenkf_atmDA — UFS ensemble Kalman filter

See `dev/ci/gitlab-ci-hosts.yml` for the complete per-machine matrices.

3.2.2 CTests (PIPELINE_TYPE=ctests)

Fast, focused unit-level testing using the CMake/CTest framework. These tests exercise individual Rocoto jobs (JJOBS) with predefined, pre-staged input data and verify their outputs against baselines from nightly stable runs.

The CTest flow:

1. `cmake -S "${GW_HOME}gefs"` — Configure the CTest build
2. `ctest -N` — List available tests
3. `ctest -L "${CTEST_NAME}"` — Run tests matching a specific label
4. JUnit XML results are published as GitLab artifacts

CTests provide rapid developer feedback (minutes instead of hours) and are ideal for targeted validation of specific job changes.

3.3 Per-Host Test Matrices

Each HPC platform runs a specific subset of test cases, defined in `dev/ci/gitlab-ci-hosts.yml`. The matrices reflect the software and data availability on each system:

Table 4: Test Cases by Platform

Platform	Test Cases
Hera	C48_ATM, C48_S2SW, C48_S2SWA_gefs, C48mx500_3DVarAOWCDA, C48mx500_hybAOWCDA, C96C48_hybatmDA, C96C48_hybatmsnowDA, C96C48_hybatmsoilDA, C96C48_ufsgsi_hybatmDA, C96C48_ufs_hybatmDA, C96C48mx500_S2SW_cyc_gfs, C96_atm3DVar, C96_gcafs_cycled, C96_gcafs_cycled_noDA, C96mx100_S2S, C48_gsienkf_atmDA, C48_ufsenkf_atmDA
Gaea C6	C48_ATM, C48_S2SW, C48_S2SWA_gefs, C48mx500_3DVarAOWCDA, C48mx500_hybAOWCDA, C96C48_hybatmDA, C96C48_hybatmsnowDA, C96C48_hybatmsoilDA, C96C48mx500_S2SW_cyc_gfs, C96_atm3DVar, C96_gcafs_cycled, C96_gcafs_cycled_noDA, C96mx100_S2S, C48_gsienkf_atmDA, C48_ufsenkf_atmDA
Orion	C48_ATM, C48_S2SW, C48_S2SWA_gefs, C96C48_hybatmDA, C96C48mx500_S2SW_cyc_gfs, C96_atm3DVar, C96mx100_S2S, C96_gcafs_cycled
Hercules	C48_ATM, C48_S2SW, C48_S2SWA_gefs, C48mx500_3DVarAOWCDA, C48mx500_hybAOWCDA, C96C48_hybatmDA, C96C48mx500_S2SW_cyc_gfs, C96_atm3DVar, C96mx100_S2S, C96_gcafs_cycled
Ursa	C48_ATM, C48_S2SW, C48_S2SWA_gefs, C48mx500_3DVarAOWCDA, C48mx500_hybAOWCDA, C96C48_hybatmDA, C96C48_hybatmsnowDA, C96C48_hybatmsoilDA, C96C48_ufsgsi_hybatmDA, C96C48_ufs_hybatmDA, C96C48mx500_S2SW_cyc_gfs, C96_atm3DVar, C96mx100_S2S, C96_gcafs_cycled, C96_gcafs_cycled_noDA, C48_gsienkf_atmDA, C48_ufsenkf_atmDA

3.4 Pipeline Variables

The following variables control pipeline behavior and can be set from GitLab scheduled pipelines, GitHub Actions triggers, or the GitLab web UI:

Table 5: Key Pipeline Variables

Variable	Default	Description
PIPELINE_TYPE	pr_cases	Testing modality: <code>pr_cases</code> or <code>ctests</code>
GFS_CI_RUN_TYPE	pr_cases	Run classification: <code>pr_cases</code> or <code>nightly</code>
RUN_ON_MACHINES	all	Space-separated list of machines or <code>all</code>
PR_NUMBER	0	GitHub PR number (0 = develop branch)
GITHUB_COMMIT_SHA	(empty)	PR head commit SHA for GitLab native GitHub integration
GW_REPO_URL	...github.com/ NOAA-EMC/...	Authoritative GitHub repository URL

4 GitHub Actions Integration

Pipelines are triggered from GitHub via the `trigger-gitlab-pipelines.yml` workflow in `.github/workflows/`. This provides a user-friendly interface for developers to initiate CI testing.

4.1 Triggering a Pipeline

1. Navigate to the **Actions** tab in the GitHub repository.
2. Select the “**Trigger GitLab Pipelines**” workflow.
3. Click “**Run workflow**” and configure the inputs:
 - **PR number:** Enter the PR number to test, or 0 for the develop branch.
 - **Pipeline Type:** Choose “PR Cases” or “CTests”.
 - **Machine checkboxes:** Select which RDHPCS machines to run on (Hera, Gaea C6, Orion, Hercules, Ursa).
4. Click “**Run workflow**” to submit.

The workflow performs the following:

1. **Permission check:** Verifies the triggering user is in the `AUTHORIZED_GITLAB_TRIGGER_USERS` list (stored as a GitHub repository variable).
2. **Parameter setup:** Resolves the PR head commit SHA, determines the pipeline type, and builds the machine selection list.
3. **GitLab trigger:** Sends a POST request to the GitLab Pipeline Trigger API with all the necessary variables.
4. **Label management:** Adds `CI-<Machine>-Ready` labels to the PR on GitHub.

4.2 Required GitHub Secrets and Variables

Table 6: GitHub Configuration

Name	Type	Description
GITLAB_TRIGGER_TOKEN	Secret	GitLab pipeline trigger token (Settings > CI/CD > Pipeline triggers)
GITHUBTOKEN	Secret	GitHub personal access token with repo scope
GW_REPO_URL	Variable	GitHub repository URL
GITLAB_TRIGGER_URL	Variable	GitLab trigger API endpoint URL
AUTHORIZED_GITLAB_TRIGGER_USERS	Variable	Comma-separated list of authorized GitHub usernames

4.3 PR Label Lifecycle

GitHub PR labels track the CI state through the pipeline:

Table 7: CI Label Flow

Label	Set By	Meaning
CI-<Machine>-Ready	GitHub Actions	Pipeline has been triggered for this machine
CI-<Machine>-Building	Build stage	Build is in progress
CI-<Machine>-Running	Build stage (on success)	Tests are actively running
CI-<Machine>-Passed	Finalize (success)	All test cases passed on this machine
CI-<Machine>-Failed	Finalize (failure)	One or more test cases failed

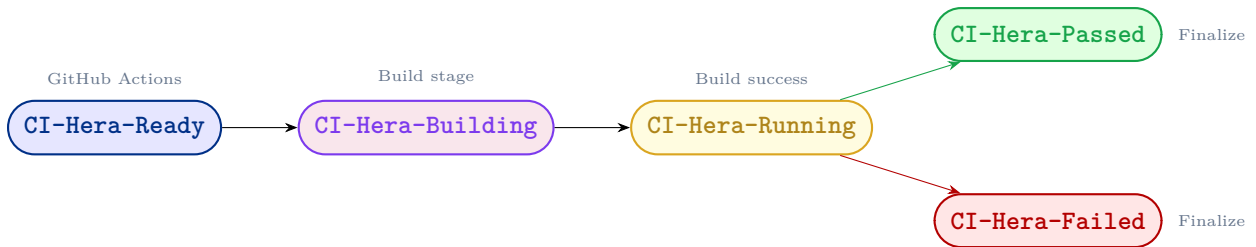


Figure 4: PR label state transitions through the CI pipeline lifecycle (Hera shown as example).

When a test case fails, the `run_check_gitlab_ci.sh` script automatically posts a comment to the GitHub PR containing:

- The failed case name and machine
- The experiment directory path
- Links to error log gists (uploaded via `publish_logs.py`)

5 Nightly Pipeline Operations

Nightly pipelines are configured as **GitLab scheduled pipelines** with `GFS_CI_RUN_TYPE=nightly`. They differ from PR-triggered pipelines in several ways.

5.1 Directory Management

On successful completion of a nightly pipeline:

1. The workspace directory is renamed from the pipeline-ID format to a date-based format:

```
# During execution:
${CI_BUILDS_DIR}/nightly_${CI_COMMIT_SHORT_SHA}_${CI_PIPELINE_ID}/

# After success:
${CI_BUILDS_DIR}/nightly_${CI_COMMIT_SHORT_SHA}_${MMDDYY}/
```

2. A **stable** symlink is created pointing to the latest successful nightly:

```
${CI_BUILDS_DIR}/stable -> nightly_${CI_COMMIT_SHORT_SHA}_${MMDDYY}/
```

3. Old nightly directories (except the stable target) are cleaned up.

The **stable** directory is significant because CTest baseline data (`STAGED_CTESTS`) is sourced from it:

```
export STAGED_CTESTS=${GITLAB_BUILDS_DIR}/stable/RUNTESTS
```

5.2 Badge Updates

Nightly pipelines update status badges stored as GitHub Gists. On success, a green “passed” badge is generated; on failure, a red “failed” badge is generated. These badges are referenced from the project README for visibility.

```
# Badge generation (from finalize stage)
curl -sSL "https://img.shields.io/badge/${machine}_nightly-passed-brightgreen" \
  -o "${badge_img_file}"
${GH} gist edit "${badge_GIST_ID}" --add "${badge_img_file}"
```

6 GitLab Runner Setup

GitLab runners are deployed directly on each RDHPCS system. They execute as **shell runners** (not Docker), running directly in the HPC environment with access to the native compilers, Spack-Stack modules, and shared filesystems.

6.1 Platform Configuration Files

Each supported platform has a configuration file at `dev/ci/platforms/config.<MACHINE_ID>` that defines platform-specific paths and settings:

Table 8: Platform Configurations

Platform	Config File	CI Root Directory
Hera	<code>config.hera</code>	<code>/scratch3/NCEPDEV/global/role.glopara/GFS_CI_CD/HERA</code>
Gaea C6	<code>config.gaeac6</code>	<code>/gpfs/f6/drda-precip3/proj-shared/\${USER}/GFS_CI_CD</code>
Orion	<code>config.orion</code>	<code>/work2/noaa/global/\${USER}/GFS_CI_CD/ORION</code>
Hercules	<code>config.hercules</code>	<code>/work2/noaa/global/role-global/GFS_CI_CD/HERCULES</code>
Ursa	<code>config.ursa</code>	<code>/scratch3/NCEPDEV/global/role.glopara/GFS_CI_CD/URSA</code>
WCOSS2	<code>config.wcss2</code>	<code>/lfs/h2/emc/global/noscrub/globalworkflow.ci/GFS_CI_ROOT</code>

Each configuration file exports the following key variables:

Listing 2: Platform configuration variables (Hera example)

```
# Base directory for all CI operations
export GFS_CI_ROOT=/scratch3/NCEPDEV/global/role.glopara/GFS_CI_CD/HERA

# Initial condition data for experiments
export ICSDIR_ROOT=/scratch3/NCEPDEV/global/role.glopara/data/ICSDIR

# GitLab runner registration URL
export GITLAB_URL=https://vlab.noaa.gov/gitlab-community

# Human-readable runner name
export GITLAB_RUNNER_NAME="RDHPCS Hera"

# Directory where pipeline builds are stored
export GITLAB_BUILDS_DIR=${GFS_CI_ROOT}/BUILDS/GITLAB

# GitLab runner working directory (state files, config)
export GITLAB_RUNNER_DIR="${GFS_CI_ROOT}/GitLab/Runner"

# Baseline data for CTests
export STAGED_CTESTS=${GITLAB_BUILDS_DIR}/stable/RUNTESTS

# Custom Rocoto path (dry-run capable build)
export GFS_CI_ROCOTO_PATH="${GFS_CI_UTIL_PATH}/src/rocoto-1.3.7-dryrun_nodaemon/bin"
```

Note

Hera and Ursa share the same physical filesystem (cross-mounted), so their `GFS_CI_ROOT` paths include the machine name (HERA or URSA) to avoid collisions.

6.2 The `launch_gitlab_runner.sh` Script

The `dev/ci/scripts/utils/gitlab/launch_gitlab_runner.sh` script is the primary tool for managing GitLab runners on each RDHPCS system. It supports three operations: **register**, **run**, and **unregister**.

6.2.1 Setup Prerequisites

Before using the launch script, ensure:

1. **Platform config exists:** A `config.<MACHINE_ID>` file must exist in `dev/ci/platforms/` for the target machine.
2. **Runner token is available:** The GitLab runner registration token must be available through one of:
 - Command-line argument (second positional parameter)
 - `GITLAB_RUNNER_TOKEN` environment variable
 - A `gitlab_token` file in the runner directory
3. **Runner binary:** The script will automatically download the GitLab runner binary if it is not present in the `GITLAB_RUNNER_DIR`.

6.2.2 Registering a Runner

To register a new runner on an RDHPCS system:

```
# SSH to the target HPC system
ssh role.glopara@hera.rdhpcs.noaa.gov

# Navigate to the global-workflow checkout
cd /path/to/global-workflow

# Register the runner (token can also be in GITLAB_RUNNER_TOKEN
# or gitlab_token file)
dev/ci/scripts/utils/gitlab/launch_gitlab_runner.sh register <GITLAB_RUNNER_TOKEN>
```

The registration command configures the runner with:

- **Executor:** `shell` (runs directly in the HPC environment)
- **Shell:** `bash`
- **Builds directory:** `${GITLAB_BUILDS_DIR}` (from platform config)
- **Custom build directory:** enabled (allowing `.gitlab-ci.yml` to override the clone path via `GIT_CLONE_PATH`)
- **Concurrency:** 24 concurrent requests

After registration, the script updates the runner's `config.toml` to set `concurrent = 24`.

6.2.3 Starting a Runner

```
dev/ci/scripts/utils/gitlab/launch_gitlab_runner.sh run
```

This launches the runner as a background process using `nohup`. The runner's working directory is set to `${GITLAB_RUNNER_DIR}` from the platform config. Logs are written to a date-stamped log file in the working directory.

6.2.4 Unregistering a Runner

```
dev/ci/scripts/utils/gitlab/launch_gitlab_runner.sh unregister
```

This removes the runner registration identified by `${GITLAB_RUNNER_NAME}` from the GitLab server.

6.3 Runner Directory Layout

Each platform follows a common directory structure under its `GFS_CI_ROOT`:

```

${GFS_CI_ROOT}/
├── BUILDS/
│   ├── GITLAB/
│   │   ├── pr_cases_<sha>_<id>/
│   │   ├── nightly_<sha>_<date>/
│   │   └── stable -> nightly_<sha>_<date>/
│   └── GitLab/
│       ├── Runner/
│       │   ├── gitlab-runner      Runner binary
│       │   ├── config.toml       Runner configuration (auto-generated)
│       │   ├── gitlab_token      Optional token file
│       │   └── launched_gitlab_runner-*.log
│       └── Jenkins/              Legacy
│           ├── agent/
│           └── workspace/

```

Figure 5: Runner directory layout on each RDHPCS platform.

6.4 Runner Maintenance

Common maintenance tasks:

Check if a runner is active:

```
ps aux | grep gitlab-runner
```

View runner logs:

```
tail -f ${GFS_CI_ROOT}/GitLab/Runner/launched_gitlab_runner-*.log
```

Restart a runner (e.g., after system maintenance):

```

# Stop any existing runner
pkill -f "gitlab-runner run"

# Start fresh
cd /path/to/global-workflow
dev/ci/scripts/utils/gitlab/launch_gitlab_runner.sh run

```

Re-register after token rotation:

```
# Unregister the old runner
```

```
dev/ci/scripts/utils/gitlab/launch_gitlab_runner.sh unregister

# Register with the new token
dev/ci/scripts/utils/gitlab/launch_gitlab_runner.sh register <NEW_TOKEN>

# Start the runner
dev/ci/scripts/utils/gitlab/launch_gitlab_runner.sh run
```

7 Pipeline Execution Details

7.1 Build Stage

The build stage (defined in `.build_template` in `.gitlab-ci.yml`) performs:

1. **Environment setup:** Sources the platform config and validates paths.
2. **Custom Rocoto loading:** If `GFS_CI_ROCOTO_PATH` is set in the platform config, it is prepended to `PATH` to use a custom Rocoto build with dry-run support.
3. **PR checkout:** For PR pipelines (`PR_NUMBER != 0`), the build fetches the PR from GitHub and checks it out using `gh pr checkout`.
4. **Build execution:** Calls `dev/ci/scripts/utils/ci_utils.sh build`.
5. **Workflow linking:** Runs `sorc/link_workflow.sh` to create necessary symlinks.
6. **Label updates:** Updates GitHub PR labels from `CI-<Machine>-Ready` to `CI-<Machine>-Building` and then to `CI-<Machine>-Running`.

7.2 Test Execution (PR Cases)

The `run_check_gitlab_ci.sh` script manages each experiment's lifecycle:

1. Launches the experiment with `rocotorun`.
2. Enters a monitoring loop that alternates between `rocotorun` and `rocotostat` calls.
3. Tracks Rocoto state through completion (`DONE`) or failure (`FAIL`, `UNAVAILABLE`, `UNKNOWN`, `STALLED`).
4. On failure: extracts error logs from failed/dead tasks, uploads them as GitHub Gists, and posts a comment to the PR.
5. Exits with `rc=0` for success or `rc=1` for failure.

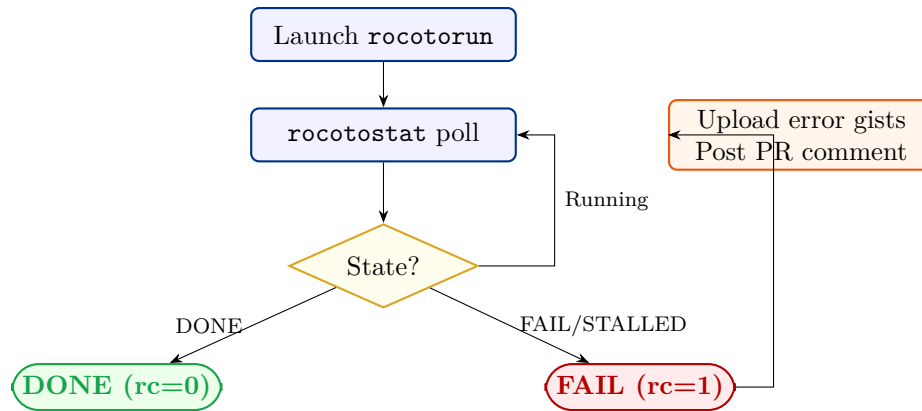


Figure 6: PR case test execution flow showing the Rocoto monitoring loop.

7.3 Test Execution (CTests)

CTest execution (defined in `.run_ctests_template` in `gitlab-ci-ctests.yml`):

1. Changes to the CTest build directory.
2. Runs `ctest -L "${CTEST_NAME}"` to execute tests for a specific label.
3. Publishes JUnit XML results as GitLab artifacts.
4. Examines both the `ctest` exit code and the JUnit XML for failure indicators.

7.4 Finalize Stage

On **success**:

- PR pipelines: Adds `CI-<Machine>-Passed`, removes `CI-<Machine>-Running`.
- Nightly pipelines: Renames the workspace to date format, creates the `stable` symlink, cleans old directories, and updates status badges.

On **failure**:

- PR pipelines: Adds `CI-<Machine>-Failed`, removes `CI-<Machine>-Running`.
- Nightly pipelines: Updates the status badge to show failure.

Failure cleanup is also handled in `after_script` blocks that run regardless of job status, canceling any remaining batch jobs and cleaning up resources.

8 Adding a New Host Platform

To extend the CI pipeline to a new RDHPCS system:

1. **Create a platform config:** Add `dev/ci/platforms/config.<new_machine>` with the required environment variables (follow an existing config as a template).
2. **Define the test matrix:** Add a case matrix in `dev/ci/gitlab-ci-hosts.yml`:

```
.new_machine_cases_matrix: &new_machine_cases
- caseName: ["C48_ATM", "C48_S2SW", ...]
```

3. **Add host-specific jobs:** Create setup, run, and finalize jobs in `dev/ci/gitlab-ci-hosts.yml` that extend the appropriate templates and reference the new machine tag:

```
setup_experiments-new_machine:
  extends: .setup_experiment_template
  variables:
    machine: new_machine
  tags:
    - new_machine
  parallel:
    matrix: *new_machine_cases
  needs:
    - build-new_machine
  rules:
    - if: $PIPELINE_TYPE == "pr_cases" && ...
```

4. **Add a build job:** Add a build job in `dev/ci/gitlab-ci-hosts.yml`:

```
build-new_machine:
  extends: .build_template
  variables:
    machine: new_machine
  tags:
    - new_machine
```

5. **Register a runner:** SSH to the new machine and register a GitLab runner using `launch_gitlab_runner.sh register`.
6. **Update GitHub Actions:** Add a new boolean input for the machine in `.github/workflows/trigger-gitlab-pipelines.yml`.
7. **Stage baseline data:** Ensure nightly baseline data is available at the `STAGED_CTESTS` path for CTest validation.

9 File Reference

Table 9: Complete File Reference

File Path	Description
<code>.gitlab-ci.yml</code>	Main pipeline orchestration and base templates
<code>dev/ci/gitlab-ci-cases.yml</code>	Setup, run, and finalize templates for experiment cases
<code>dev/ci/gitlab-ci-ctests.yml</code>	CMake/CTest setup and execution templates

File Path	Description
<code>dev/ci/gitlab-ci-hosts.yml</code>	Per-host job definitions, test matrices, and runner tags
<code>dev/ci/platforms/config.*</code>	Platform-specific CI/CD environment configuration
<code>dev/ci/cases/pr/*.yaml</code>	Individual test case definitions (experiment YAML files)
<code>dev/ci/scripts/utils/ci_utils.sh</code>	Core CI utility functions (build, create_experiment, etc.)
<code>dev/ci/scripts/run_check_gitlab_ci.sh</code>	Experiment monitoring, Rocoto polling, and failure reporting
<code>dev/ci/scripts/utils/gitlab/launch_gitlab_runner.sh</code>	GitLab runner registration, startup, and removal
<code>dev/ci/scripts/utils/gitlab/badge-updater-pipeline.yml</code>	Standalone badge update pipeline configuration
<code>dev/ci/scripts/utils/publish_logs.py</code>	Error log upload to GitHub Gists
<code>dev/ci/scripts/utils/rocotostat.py</code>	Rocoto status parsing and reporting
<code>.github/workflows/trigger-gitlab-pipelines.yml</code>	GitHub Actions workflow for triggering GitLab pipelines

10 Troubleshooting

10.1 Runner Not Picking Up Jobs

1. Verify the runner process is active: `ps aux | grep gitlab-runner`
2. Check runner logs for connection errors.
3. Ensure the runner tags match the job tags in the pipeline configuration.
4. Verify network connectivity to the GitLab instance from the HPC node.

10.2 Build Failures

1. Check that `GW_HOMEgfs` is correctly set and the directory exists.
2. Verify that Spack-Stack modules are loadable on the target platform.
3. Review the `ci_utils.sh` build output in the job logs.
4. For PR builds, ensure `gh` (GitHub CLI) is installed and authenticated.

10.3 Test Case Timeouts

1. Rocoto-based experiments have a maximum Rocoto cycle timeout configured in the CI runner (`RUNNER_SCRIPT_TIMEOUT: 8h`).
2. If experiments consistently time out, check:
 - Job scheduler queue availability on the HPC system.

- `maxtries` setting in the test case YAML.
- Whether batch jobs are being submitted and scheduled correctly.

10.4 CTest Baseline Mismatches

1. Verify that `STAGED_CTESTS` points to a valid, recent nightly build.
2. Confirm the `stable` symlink is intact and pointing to a successful nightly.
3. Check that the baseline data matches the current develop branch state.

10.5 GitLab Mirror Sync Issues

1. Verify the pull mirror is operational on the licensed GitLab instance (Settings > Repository > Mirroring repositories).
2. Check the “Last successful update” timestamp — it should be within the last few minutes.
3. For push mirror issues to the community instance, verify the credentials and target URL are still valid.
4. If a specific branch is missing, trigger a manual sync from the mirroring settings page.